

Security-Aware Deployment Optimization of Cloud–Edge Systems in Industrial IoT

Valentina Casola^{ID}, Alessandra De Benedictis^{ID}, Sergio Di Martino^{ID}, Nicola Mazzocca,
and Luigi Libero Lucio Starace^{ID}

Abstract—Cloud computing, edge computing, and the Internet of Things are significantly changing from the original architectural models with pure provisioning of virtual resources (and services) to a transparent and adaptive hosting environment, where cloud providers, as well as “on-premise” resources and end nodes, fully realize the “everything-as-a-service” provisioning concept. The optimal design of these architectures, including the selection of optimal services to acquire, is not trivial in the cloud–edge context due to the involvement of a variable number and the type of available resources offerings and to the impact on cost, performance, and other relevant features such as security, almost never considered. This article presents a novel formalization of the cloud–edge allocation problem for the industrial IoT context. The proposed optimization process takes explicitly into account **two critical aspects that are often overlooked in similar approaches, namely, the new cloud–edge on-demand service offerings model for the allocation of resources and the impact on the deployed application, in terms of cost, performance, and security policies actually implemented.** An efficient yet suboptimal deterministic solver is also presented and compared with a linear programming one. Results are the same in 86% of the cases on the considered data set while our solver is orders of magnitude faster than the linear one.

Index Terms—Cloud–edge architectures, cloud–edge service deployment optimization, secure Industrial Internet-of-Things (IIoT) application development.

I. INTRODUCTION

THE Internet of Things (IoT) is one of the most dynamic and promising areas in the current IT landscape based on the idea that digitally interconnected physical objects can continuously sense contextual information without human intervention [1]–[3]. When IoT is applied to the Industrial scenario, it is known as *Industrial IoT* or IIoT, whose goal is to collect status data and control industrial machinery. Indeed, the combination of IIoT along with big data analytics is opening great possibilities to develop innovative, distributed software applications to increase business productivity [4].

The spreading of IoT/IIoT is leading to the constitution of a huge, connected, and heterogeneous digital environment, where one trillion interconnected sensors are foreseen by 2030 [5], generating data at an unprecedented rate. In the industrial context, over four trillion gigabytes of data can be

generated in a year by IIoT devices [6], and this volume will increase [7]. This is a great challenge for information and communication technology practitioners [8]. One of the key technologies to handle huge amounts of data is *cloud* computing, where, thanks to the adoption of the on-demand leasing model, customers can acquire massive services, computational and storage resources from a provider, and pay only for the capacity they actually use, thus *cloudifying* their applications.

Anyhow, in the IIoT scenario, solutions purely cloud based may have some drawbacks since the requirements in terms of response time, network latency, and reliability are especially critical [9]. Moreover, these problems can also get worse due to the sensitive nature of data and to company requirements, willing to nondisclose all the data with the cloud provider, requiring to carefully take into account the security features offered in the cloud. *Edge* computing can mitigate these problems by offering computational and storage resources lying between IIoT devices and cloud data centers [10]. An edge device has indeed some computational power and storage and lesser network-related problems with respect to a data center. Thus, the cloud–edge computation paradigm (CE in the following) recently emerged as a valuable solution for IIoT [9], [11]. Indeed, this paradigm distributes part of the intelligence, processing power, and communication capabilities: the so-called edge devices, highly-heterogeneous appliances close to the IIoT devices, perform preliminary processing with very low latency and the highest security standards of the company, while heavy computation tasks are carried out in cloud data centers.

The CE computational paradigm presents two key challenges: 1) the identification of a suitable deployment distribution of software components between edge and cloud and 2) the identification of the most effective cloud provider for the intended services. Indeed, cloud vendors offer a very wide portfolio of products, each characterized by different computation, memory, networking capabilities, security properties, and, of course, costs. Consequently, the risk of incurring overspending or of failing to respect some requirements due to a poor choice is considerably high. As a consequence, nowadays, an IIoT solution architect, in charge of choosing architecture and a cloud provider to handle IIoT data streams, might be in trouble in identifying the most suitable solutions due to the overwhelming number of variables to be considered in this allocation.

Many papers have been presented in the literature to help practitioners in facing the problem of how to allocate

Manuscript received March 30, 2020; revised June 13, 2020; accepted June 19, 2020. Date of publication June 24, 2020; date of current version August 6, 2021. (Corresponding author: Valentina Casola.)

The authors are with the Department of Electrical Engineering and Information Technology, University of Naples Federico II, 80125 Naples, Italy (e-mail: casolav@unina.it).

Digital Object Identifier 10.1109/IIOT.2020.3004732

components of a distributed application to cloud providers (e.g., [12]). Nevertheless, to the best of our knowledge, among the several cloud allocation optimization strategies proposed in the literature, trying to optimize the placement of cloud services on virtual machines, the distribution over a wider CE paradigm and the consideration of security requirements have received very little attention. Some recent research projects (such as SPECS [13] and MUSA [14]) have promoted the adoption of security service-level agreements (SLAs) [15], [16] to declare the security capabilities offered on top of cloud services, but never in a CE context.

To fill this gap, in this article, we propose a novel formalization of the cloud-edge allocation problem in the context of IIoT that leverages a component-based description of an application to enable the identification of optimal CE allocation strategy for the application components, taking into account cost, performance, and security aspects. In order to cope with the size of the problem, which can easily grow to nonfeasible levels, even with a relatively small number of components and instance types, we present also a deterministic algorithm able to find a suboptimal solution in milliseconds. This algorithm has been empirically compared against a solver based on linear programming, which required many days of computation. Results show that in 86% of the cases, our algorithm found the same allocation of the linear solver, but in a fraction of the required computational time.

Summarizing, the main contributions of this article are as follows.

- 1) We bring to the attention of the research community the issues related to the security-aware cloud-edge allocation problem, in the context of industrial IIoT.
- 2) We provide a formalization of the problem, taking into account the distribution between cloud and edge, cost, performance, and security aspects.
- 3) We describe an efficient solver for this problem, able to find quickly suboptimal solutions, and we experimentally compare it against a linear programming solver.

The remainder of this article is organized as follows. In Section II, we describe the related work. In Section III, we provide the formalization of the security-aware CE allocation problem. In Section IV, we describe the computational issues related to the solution of the problem, present the efficient solver we devised, and report on the results of the experimental evaluation. In Section V, some discussion and final remarks conclude this article.

II. RELATED WORKS

The optimal allocation problem in the cloud/edge is faced in the literature from different perspectives. The available approaches take into account different deployment models that lead to different formulations of the problem and solving algorithms. Nevertheless, in most of them, the cloud application is typically seen as a multitask, and the problem is traditionally formalized as a bin-packing/multicriteria optimization problem and solved by using various tools implementing heuristics, such as greedy, linear programming, and dynamic programming.

Many systematic reviews, such as [17]–[23] introduce a very interesting characterization of cloud/edge resource allocation goals and parameters. In particular, the parameters identified are as follows.

- 1) Cost-aware resource allocation.
- 2) Efficiency-aware resource allocation.
- 3) Load balancing-aware resource allocation.
- 4) Power-aware resource allocation.
- 5) QoS-aware resource allocation.
- 6) Utilization-aware resource allocation.

It is worth noting that none of the surveyed papers explicitly focused the attention on security. Furthermore, in the majority of the analyzed approaches, the problem is modeled as a simple allocation problem with the distribution of n tasks over a set of m virtual machines (and k physical machines) without taking into considerations all the parameters that a cloud service provider, today, offers as, for example, memory capabilities, network bandwidth, storage space, outbound traffic, etc.

This context is further complicated by the possibility of using cloud services from different providers, and by the need of considering on-premise edge devices as well as latency and security constraints. Thus, resource provisioning for IIoT applications can be a challenging task.

Casola *et al.* [15] proposed a security SLA model for cloud applications to formally quantify the security level provided by a cloud application, deployed in a multicloud environment, to optimize the planning of needed resources in order to guarantee a given security level, the main innovation proposed by this article was the security model to be adopted in the optimization algorithm. The optimization process proposed in [24] also adopted a quantitative approach based on security SLAs and it took explicitly into account two critical aspects, namely, the cloud on-demand leasing model for the allocation of resources and the impact that the deployment has on the security policies actually implemented by a complex cloud application. The main innovation proposed by this article was the multicriteria optimization algorithm that included security parameters and some real cloud constraints.

More recently, with the emergence of IIoT and the fog computing paradigm, which extends the cloud services to the edge of the network, allowing for computation, communication, and storage to be closer to end users, many works focused on provisioning and deployment in fog scenarios.

Djemai *et al.* [25] addressed the problem of placing IIoT services over a fog computing infrastructure, combining both cloud data centers, edge, and IIoT devices. This work focuses on the minimization of energy consumption and delay constraints violations using a discrete particle swarm optimization approach. In a similar context, Mishra *et al.* [26] proposed three biobjective heuristic algorithms aimed at allocating services in a fog computing environment while minimizing both energy consumption and the overall required computation times.

Battula *et al.* [27] proposed a detailed cost model and a resource allocation method for fog environments, taking into account connectivity costs as well as data transfer and processing costs.

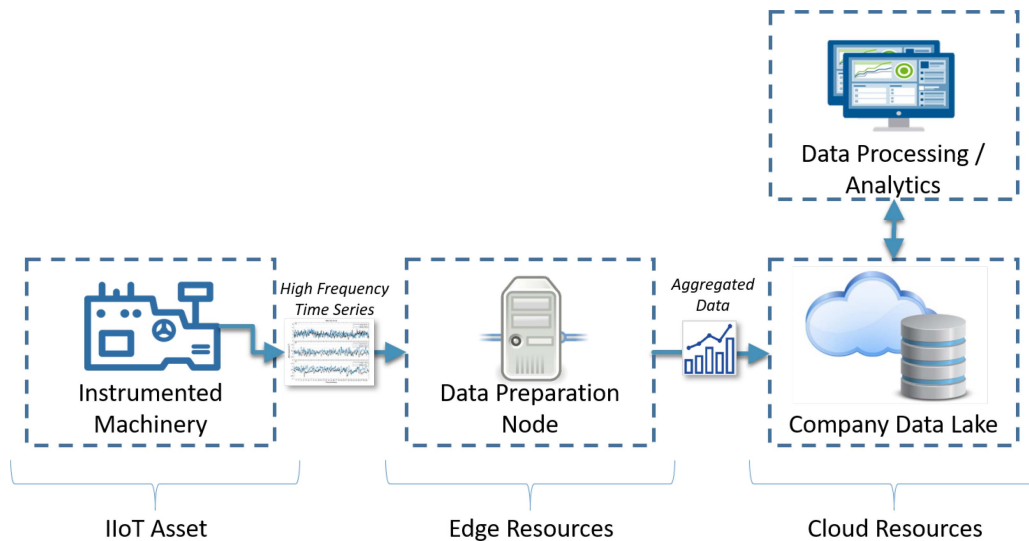


Fig. 1. Reference architecture for the investigated IIoT scenario.

Problema fondamentale
del paper

Benamer *et al.* [28] contributed a study on how to deploy an IoT application, modeled as a set of interconnected components, on edge devices while minimizing the overall of the application. They formalize the optimization problem as an integer linear programming problem and also propose a heuristic to efficiently compute near-optimal deployments.

Dehnavi *et al.* [29] proposed a resource provisioning scheme for real-time IIoT applications, taking into account reliability issues as well as latency constraints. In this work, a number of tasks are first mapped to virtual machines, and then such virtual machines are mapped to either edge servers or private/public cloud services.

These works, however, do not take into account data transfer costs, which in IIoT scenarios are not negligible at all, and also ignore security and latency constraints which likely arise in IIoT scenarios.

III. OPTIMAL CLOUD-EDGE ALLOCATION PROBLEM

The reference IIoT cloud-edge architecture investigated in this article is presented in Fig. 1. With reference to the figure, we have the *instrumented machinery* under monitoring, whose sensors produce multiple high-frequency time series. Due to the amount [30] and the sensitivity of these collected data, sending them all to the cloud might not be a viable solution. Consequently, some local computational resources can help to face these network/security-related problems. Thus, in our considered architecture, the IIoT asset is locally connected to a *data preparation node*, which might be a personal computer with some computational and storage resources. This represents the edge layer of our architecture. This data preparation node is typically located inside the factory LAN in order to reduce latency for local analytics tasks. Data are then sent to the cloud for further massive storage and processing such as, for example, for data analytics tasks. The data transfer to the cloud can be also done when the load of the network is lower such as during the night.

Given this reference architecture, the cloud-edge allocation problem tackled in this article consists in identifying the optimal distribution of the components of company services/applications onto the edge and the remote cloud-hosted computing resources so that performance, security, and latency constraints are satisfied, and the overall quality of the obtained deployment results optimal according to specific performance, security, or cost criteria.

In this article, we assume that the IIoT solution to deploy is made of several interconnected logic components, either custom or already available as commercial-off-the-shelf (COTS), and that these components can be in principle installed and executed on any virtual or physical machine (including the edge devices), provided that minimum requirements are satisfied. In particular, we assume that each component can be characterized by specific requirements in terms of latency, computation, networking, and storage capabilities, which are known to the solution architect, based on the components behavior and interconnections and on the expected workload.

Moreover, we also assume that a security profile is known for each component, consisting of the set of security controls that the running component should fulfill to mitigate security risks. Security controls refer to both technical and organizational measures that enable to avoid, detect, counteract, or minimize security risks to an asset, and may refer to different aspects, such as access control, identity management, security testing and monitoring, change management, etc. We assume that the security profile of a component is obtained from an offline threat modeling and risk assessment process (which is currently a well-established practice in many enterprises). Since our focus is on the security-aware deployment of components, we assume that the proper technical solutions needed to enforce a given control as part of the internal behavior of a component (e.g., user authentication, encryption of data, etc.) are correctly in place before application deployment, both in the case of custom and COTS components.

Given the security profile of a component, we refer to its security requirements as the set of controls belonging to its

security profile, whose enforcements depend on the security features of the resources on which it will be deployed. In some cases, these features may be inherently enforced by the deployment resources (e.g., in the case when a cloud instance is equipped with vulnerability scanning tools or continuous monitoring tools, or in case of an on-premise device provided with a trusted platform module). In other cases, the above features may be related to more organizational aspects concerning the general security policies enforced by the provider (e.g., strong authentication required to access the provider's data centers) or the enterprise.

A. Problem Formalization

In the following, we will formalize the problem by defining its inputs, outputs, and constraints. We will discuss three different optimization goals that provide potentially different points of view to the solution architect.

1) *Problem Inputs*: The inputs to our problem are the specification of: 1) the components forming the IIoT application to deploy and 2) the potential deployment resources.

As anticipated, the IIoT application specification is formalized by defining a set $C = \{c_1, \dots, c_n\}$ of n interconnected software components, while available deployment resources are represented by a set $O = \{o_1, \dots, o_m\}$ of m different computing resources, referred to as *offerings*. In particular, offerings are represented by either cloud-based IaaS instances (i.e., virtual machines leased by cloud providers) or local edge devices available within the company. Each component and offering has performance, network, security, and cost-related attributes, as detailed in the following.

Let us first discuss the inputs related to *offerings* and concerning performance, location, cost, and security aspects, respectively.

Offerings Performance, Networking, and Latency-Related Inputs: With regard to cloud-based *offerings*, popular cloud providers currently offer several different virtual machine types characterized by different computing capabilities expressed mainly in terms of CPU power, RAM capacity, and bandwidth capabilities. These features and their associated measures are typically discretized into small sets of ordered values defining the level at which the capability is offered. For instance, the CPU power available on a machine is defined in terms of the number of virtual or physical CPU cores, which, for example, can assume the values 1, 2, 4, 8, or 16. In this article, we assume that this characterization can be applied also to the offerings corresponding to on-premise edge devices in order to treat all deployment resources in a homogeneous way.

In light of the above, we assume that a generic offering o_j is characterized by the following parameters.

- 1) $vCPU(o_j)$ is a positive integer representing the number of virtual/physical CPU cores available on the offering, belonging to a discrete set $vCPUcores$ of available configurations.
- 2) $RAM(o_j)$ is a positive integer representing the offering RAM capacity (e.g., in gigabytes), belonging to a discrete set $RAMcapacities$ of available capacities.

- 3) $OutB(o_j)$ is a positive integer representing the offering maximum outbound data traffic rate (e.g., in gigabits per second), belonging to a discrete set $Rates$ of available rates.

- 4) $isLocal(o_j)$ is a boolean value, equal to 1 if o_j is a local resource (i.e., an on-premise edge device ensuring low latency), 0 otherwise.

Offerings Cost-Related Inputs: The actual capabilities of an offering, in terms of the above parameters, clearly influence its cost of ownership. This is true for both cloud-based instances and on-premise equipment. In particular, with regard to cloud-based leased instances, major providers adopt complex billing strategies, but a common feature is to bill separately the hourly/monthly instance usage and the amount of transferred data. VM pricing depends on the VM uptime, starting from the time it is launched until it is terminated or stopped. With regard to bandwidth pricing, the ingress traffic is typically free of charge, while the egress traffic usually depends on the source and destination location (geographic regions). Typically, providers do not charge for the traffic generated between two instances in the same region. With regard to on-premise resources, they may be already part of the enterprise assets or not. Anyhow, the *cost of ownership* (which includes the cost of electricity, refrigeration, maintenance, depreciation, etc.) is not null, and in some cases, it may be quite high.

Hence, the cost-related parameters associated with offerings are the following.

- 1) $OwnCost(o_j)$ is the cost of ownership of the offering o_j (valid for both on-premise and cloud-based resources) per unit of time (computed on a hourly or monthly basis, for example).
- 2) $ODTCost(o_j, o_k)$, $k = 1, \dots, m$, are the unitary costs per gigabyte of transferred data from offering o_j to offering $o_k \forall o_k \in O$.

Offerings Security-Related Inputs: Finally, as mentioned before, each offering has typical security features implemented by means of technical or organizational measures taken by the provider. In particular, our model assumes that the security features of an offering o_j are expressed in terms of standard security controls belonging to a reference framework (e.g., the NIST security control framework [31]).

It is worth noting that some security controls can be enforced at different levels, namely, by adopting different technologies and mechanisms, with a resulting different efficacy. For example, the IA-2 IDENTIFICATION AND AUTHENTICATION control [31] may be enforced by implementing different mechanisms ranging from the simple username/password authentication to token-based, biometrics-based, and multifactor authentication, which is characterized by an increasing level of security.

Let $SC = \{sc_1, \dots, sc_S\}$ be the set of S considered security controls and $LEV = \{0, l_1, \dots, l_L\}$ be the set of L possible implementation levels, which are assumed to be applicable to all controls. Let LEV be a subset of $\mathbb{N}_0$ with $l_i < l_k$ if $i < k$.

For example, referring to the IA-2 IDENTIFICATION AND AUTHENTICATION control mentioned above and assuming that $LEV = \{0, 1, 2, 3, 4\}$, the five different integers may correspond to the following authentication mechanisms

in place: level 0: no authentication, level 1: login/password, level 2: two-factor authentication, level 3: white-list of IPs plus credentials, and level 4: the combination of two or more mechanisms (IP, port number, certificate or public keys, credentials, etc.).

Finally, let $ProvidedLevel : SC \times O \mapsto LEV$ be a function that maps each security control sc_t and each offering o_j to the corresponding implementation level provided by o_j for the control sc_t . A value $ProvidedLevel(sc_t, o_j) = 0$ means that the control sc is not enforced at all.

The overall security characterization of o_j can be formalized as the set of all considered security controls along with associated implementation levels at o_j

$$SecSLA(o_j) = \{(sc_t, ProvidedLevel(sc_t, o_j)) : sc_t \in SC\}.$$

Let us now discuss *component requirements*, related to performance, networking, latency, and security aspects.

Components Performance, Networking, and Location-Related Inputs: It is worth noting that before selecting any offering type, a system administrator should first assess the requirements of his/her application and components and identify, as a first step, the minimum needed computation and networking capabilities. Such performance-related requirements can be formalized as follows for a generic component c_i .

- 1) $vCPUReq(c_i)$ is a positive real number identifying the minimum CPU requirements of a component based on its internal implementation and on the degree of parallelism of its internal tasks.
- 2) $RAMReq(c_i)$ is a positive real number representing the minimum RAM capacity (e.g., in gigabytes) required by the component to run smoothly.
- 3) $OutBReq(c_i)$ is the minimum required data transfer rate (e.g., in gigabits per second) of the component.

Furthermore, a component c_i has specific data transfer volume requirements, which is formalized as follows.

- 1) $DT(c_i, c_j)$, $j = 1, \dots, n$, where $DT : C \times C \mapsto \mathbb{R}_0^+$ is a function that returns the nonnegative average data traffic in gigabytes exchanged from c_i to c_j in the considered unit of time (in compliance with the computation of the cost of ownership), with $c_i, c_j \in C$ and $DT(c_i, c_i) = 0$.

Finally, let us recall that in our model, components' requirements do not include a specific desired geographical location for deployment, while we explicitly take into consideration the need for local resources to satisfy possible low-latency requirements. As remarked before, in fact, a fundamental aspect that should be taken into account in contexts such as IIoT, and which represents one of the leading factors pushing toward the adoption of a cloud-edge paradigm, is the need to control the latency for a subset of time-critical tasks. These tasks and the software components executing them are usually allocated on local machines in order to keep the latency limited. This requirement can be formalized by means of the following function.

- 1) $reqLocal(c_i)$, is a boolean value, equal to 1 if the component c_i has strict latency requirements and must be deployed on local resources (i.e., on edge devices), 0 otherwise.

Components Security-Related Inputs: Finally, with regard to the components' security profile, it can be specified, as done for instances, in terms of security controls and associated implementation levels. More specifically, a component's security profile can be specified in terms of the minimum implementation level required for each of the considered security controls. Let $requiredLevel : SC \times C \mapsto LEV$ be a function mapping each security control sc_t and each component c_i to the minimum implementation level required by $c_i \in C$ with respect to sc_t . The security profile of component c_i can be formalized as

$$SecReq(c_i) = \{(sc_t, requiredLevel(sc_t, c_i)) : sc_t \in SC\}.$$

A value $requiredLevel(sc_t, c_i) = 0$ means that control sc_t is not required by c_i .

2) *Problem Outputs:* The solution to the cloud-edge allocation problem can be seen as a double assignment of components to computing resources and of computing resources to specific offerings. More formally, given the inputs described above, a possible solution to the cloud-edge allocation problem is represented by a partition $P = \{p_1, \dots, p_k\}$ of C such that each element $p_l \in P$:

- 1) represents a subset of application components to be deployed on the same resource, which we call a *deployment set*;
- 2) is mapped to a specific offering in O by means of a function $\lambda : P \mapsto O$.

Hence, a generic solution to the optimal cloud-edge allocation problem is represented by the pair $s = \langle P, \lambda \rangle$.

To provide a concrete example, if $C = \{c_1, c_2, c_3, c_4\}$ and $O = \{o_1, o_2\}$, a possible solution may be given by $s = \langle P, \lambda \rangle$, where $P = \{p_1, p_2, p_3\}$ with $p_1 = \{c_1, c_3\}$, $p_2 = \{c_2\}$, and $p_3 = \{c_4\}$ and $\lambda(p_1) = o_1$, $\lambda(p_2) = o_2$, and $\lambda(p_3) = o_1$.

3) *Problem Constraints:* A generic solution $s = \langle P, \lambda \rangle$ is admissible if existing performance and security constraints are satisfied. In particular, assuming that the computation power, the memory capacity, and the data transfer rate are additive parameters, the following conditions must hold.

- 1) *Performance Constraints:*

$$\forall p_l \in P, \sum_{c_i \in p_l} vCPUReq(c_i) \leq vCPU(\lambda(p_l)) \quad (1)$$

$$\forall p_l \in P, \sum_{c_i \in p_l} RAMReq(c_i) \leq RAM(\lambda(p_l)) \quad (2)$$

$$\forall p_l \in P, \sum_{c_i \in p_l} OutBReq(c_i) \leq OutB(\lambda(p_l)). \quad (3)$$

Moreover, as for the latency constraints, they can be expressed as follows.

- 2) *Latency Constraints:*

$$\forall p_l \in P, \forall c_i \in p_l, reqLocal(c_i) \leq isLocal(\lambda(p_l)). \quad (4)$$

Finally, security constraints can be formalized as follows.

- 3) *Security Constraints:*

$$\forall p_l \in P, \forall c_i \in p_l, \forall sc_t \in SC \\ providedLevel(sc_t, \lambda(p_l)) \geq requiredLevel(sc_t, c_i). \quad (5)$$

Constraints (1), (2), and (3) require, respectively, that the sum of the computing, memory, and networking resources required by each component assigned to a given machine/device does not exceed the amount of respective resources provided by the offering to which the machine/device belongs. For instance, in the case of the example solution mentioned above, since c_1 and c_3 belong to the same deployment set p_1 and such deployment set is represented by offering o_1 , the overall CPU power available on o_1 must be enough to satisfy the overall CPU request of components c_1 and c_3 .

Constraint (4) requires that if a component c_i has strict latency requirements (and this is expressed by posing $\text{reqLocal}(c_i) = 1$), then it must be deployed on a local device [i.e., a deployment resource whose corresponding offering o_j is a local one, expressed with $\text{isLocal}(o_j) = 1$]. Referring to the example solution, if $\text{reqLocal}(c_1) = 1$, then the offering o_1 selected for its deployment and corresponding to p_1 must have $\text{isLocal}(o_1) = 1$, i.e., it must be a local resource.

Security constraint (5), finally, ensures that a generic deployment resource is only assigned to offerings that satisfy the minimum security-level requirements of each of the hosted components. Said in other words, if a component c_i is hosted on a machine/device represented by an offering o_j , the machine/device must offer all the controls required by the component with an implementation level at least equal to that required by the component.

4) *Optimization Goals*: As anticipated, we aim at identifying the solution to the cloud-edge allocation problem that best fits existing requirements while optimizing the overall *quality*. It is worth mentioning that optimization goals may be diverse based on the specific application domain, on the nature of processed data and information, and on the specific business requirements. In this article, we consider two different points of view that are often conflicting, namely, cost and security. In the following, we will introduce the basic metrics for cost and security and present three different objective functions related to different optimization goals.

Given a solution $s = \langle P, \lambda \rangle$ to the cloud-edge allocation problem, we define its overall *cost of ownership* per unit of time as the sum of hourly costs of the offerings belonging to the solution, formalized as

$$\text{own_cost}(s) = \sum_{p_l \in P} \text{OwnCost}(\lambda(p_l)). \quad (6)$$

The overall data transfer cost of s per unit of time depends on the amount of traffic exchanged among the machines/devices of the solution and is defined as

$$\text{dt_cost}(s) = \sum_{p_l \in P} \sum_{p_m \in P, m \neq l} \left(\sum_{c_i \in p_l} \sum_{c_j \in p_m} (DT(c_i, c_j) \times \text{ODTcost}(\lambda(p_l), \lambda(p_m))) \right). \quad (7)$$

In the above equation, the inner double summation computes the cost of exchanging traffic between any two components

residing on two different machines/devices of the solution by multiplying the amount of gigabytes transmitted from c_i to c_j by the unitary transmission cost from p_l to p_m . The outer summation considers any pair of machines/devices in the solution, thus providing the overall transfer cost.

The total cost for s is then defined as

$$\text{tot_cost}(s) = \text{own_cost}(s) + \text{dt_cost}(s). \quad (8)$$

With regard to security, we define the security score of s in terms of the *extra security* that a generic solution is able to grant provided that the security requirements are satisfied. This extra security depends on the implementation level of all considered controls and, in particular, on the (positive) difference between the offered level and the required level for each control. The security score can be formalized as follows:

$$\text{sec_score}(s) = \sum_{p_l \in P} \sum_{c_i \in p_l} \sum_{sc_t \in SC} (\text{providedLevel}(sc_t, \lambda(p_l)) - \text{requiredLevel}(sc_t, c_i)). \quad (9)$$

In light of the above, we can define three different objective functions, depending on the specific goal of the cloud application owner and on existing constraints based on the considered domain, as follows.

- 1) Minimize the overall solution cost $\text{tot_cost}(s)$.
- 2) Maximize the solution security score $\text{sec_score}(s)$.
- 3) Maximize the quality of the solution based on the multicriteria objective function defined as follows:

$$\text{obj}(s) = -(1 - \alpha)(\text{tot_cost}(s)) + \alpha \cdot \text{sec_score}(s). \quad (10)$$

It is worth noting that all the three above objectives are of interest and are worth investigation, and in all the cases, the optimization problem is very complex. However, in this article, we focus on the cost minimization objective and propose a greedy algorithm to solve it in a reasonable time, as discussed in detail in the next section.

IV. SOLVING THE SECURITY-AWARE CLOUD-EDGE ALLOCATION PROBLEM

The security-aware cloud-edge allocation problem is an instance of nonlinear integer programming, which is an NP-complete problem [32], and exhibits a very fast-growing number of possibly admissible solutions. Consider a given problem instance counting n components and m offerings. The number of distinct partitions (i.e., possible deployment resource assignments) of n components is given by the n th Bell number

$$B_n = \sum_{i=1}^n \left\{ \begin{matrix} n \\ i \end{matrix} \right\}$$

where $\left\{ \begin{matrix} n \\ i \end{matrix} \right\}$ is the Stirling number of the second kind, representing the number of partitions of a set of n elements into i nonempty subsets. Each of the k deployment resources to which the components are mapped can then be assigned to at most m distinct offerings. Therefore, the number of admissible solutions to the instance is superiorly bounded by

$$\sum_{i=1}^n \left\{ \begin{matrix} n \\ i \end{matrix} \right\} \cdot m^i.$$

Algorithm 1: solver Algorithm

```

1 Function solver( $C, O$ ):
2    $n \leftarrow |C|$ ;
3   for  $i \in \{1, \dots, n\}$  do
4      $DS[i] \leftarrow \emptyset$   $\triangleright$  Init. potential deployment sets
5   foreach  $c \in C$  do
6     for  $i \in \{1, \dots, n\}$  do
7        $off \leftarrow \text{findOffering}(O, DS[i] \cup \{c\})$ ;
8       if  $off$  is not nil then
9          $DS[i] \leftarrow DS[i] \cup \{c\}$ ;
10         $\lambda(DS[i]) \leftarrow off$ ;
11        break;
12    $P \leftarrow \{DS[i] \mid DS[i] \neq \emptyset, i \in \{1, \dots, n\}\}$ ;
13   return ( $P, \lambda$ );

```

To better grasp the potential size of the problem solution space, consider the following example. With a medium-sized problem instance involving 20 components and 50 offerings, by applying the formula above it results that there can be at most approximately 2.15×10^{35} admissible solutions.

Because of this very large-growing solution space, finding the optimal solution with a *generate-and-test* approach is not feasible for nontrivial instances. As a consequence, we devised a computationally efficient yet suboptimal greedy solution to the problem at hand. In particular, we propose a greedy-algorithm which computes a solution in polynomial time.

A. Greedy Algorithm

The proposed solution is detailed in Algorithm 1 and described as follows. The algorithm takes as input an instance of the security-aware cloud-edge allocation problem, i.e., a set of *components* to be deployed and a set of available *offerings*. In lines 3 and 4, the data structure representing the potential component partition DS , i.e., the potential assignment of components to deployment sets (see Section III-A2), is initialized to n empty sets, where n is the number of components to be deployed. Note that a solution to the security-aware cloud-edge allocation problem cannot involve more than n deployment sets, as in the worst case, each component will be assigned to a different deployment resource of its own. After the initialization, the loop in lines 5–11 iterates over each component $c \in C$ and assigns it to a deployment resource in a greedy first-fit fashion, i.e., by selecting the first compatible deployment resource it finds. In particular, to do so, the algorithm iterates over each potential deployment set $DS[i]$ (loop in lines 6–11) and tries to add c to it. If the resulting deployment set $DS[i] \cup \{c\}$ can be deployed on any of the available offerings, i.e., there exists at least one offering that satisfies all of its security and performance requirements (see Section III-A3), then c is added to the i th deployment set $DS[i]$ (line 9), and $DS[i]$ is assigned to the cheapest compatible offering off (line 10). The function `findOffering` (line 7) takes as input the set of available offerings and a subset of components C' , and

greedily returns the cheapest offering that satisfies all the security, latency, and performance constraints of the components in C' , or *nil* if there is no compatible offering. More formally, the `findOffering` function selects the offering o_j having the minimum $OWNCost(o_j)$ among those that satisfy all the constraints of C' , i.e., the set

$$\left\{ \begin{array}{l} o_j \in O \text{ s.t.} \\ \sum_{c_i \in C'} \text{vCPUReq}(c_i) \leq \text{vCPU}(o_j) \wedge \\ \sum_{c_i \in C'} \text{RAMReq}(c_i) \leq \text{RAM}(o_j) \wedge \\ \sum_{c_i \in C'} \text{OutBReq}(c_i) \leq \text{OutB}(o_j) \wedge \\ \forall c_i \in C', \text{reqLocal}(c_i) \leq \text{isLocal}(o_j) \wedge \\ \forall (c_i, sc_i) \in C' \times SC, \\ \text{isLevel}_{o_j}(sc_i) \geq \text{reqLevel}_{c_i}(sc_i) \end{array} \right\}.$$

To do so, it iterates over the offerings in an ascending cost order, returning the current offering as soon as a compatible offering is found.

In line 12, the actual partition of components in the deployment set is computed by selecting the nonempty potential deployment sets and stored in P .

B. Complexity Analysis

Let n and m be, respectively, the number of components and offerings in a given instance of the security-aware cloud-edge allocation problem. The greedy solver we propose computes a solution in polynomial time $O(n^2 \cdot (m + n))$, as we detail in the following complexity analysis. The first loop in lines 3 and 4 in Algorithm 1 performs n constant-time initialization operations. The loops in lines 5 and 6 iterate each at most n times, thus lines 7–11 are executed at most n^2 times. Line 7 can be executed in time $O(n+m)$ since the overall requirements for the set of components $DS[i] \cup \{c\}$ can be computed by iterating over the involved components in time $O(n)$, and a compatible offering (if any) can be found by iterating over each offering in $O(m)$. Since lines 8–11 can be executed in constant time, the overall time complexity for the loop in lines 5–11 is $O(n^2 \cdot (m + n))$. Finally, line 12 can also be executed in time $O(n)$. Therefore, the overall time complexity of the algorithm is $O(n) + O(n^2 \cdot (m + n)) + O(n) = O(n^2 \cdot (m + n))$.

C. Correctness Proof

In this section, we show that Algorithm 1 is correct, i.e., if the given input instance is solvable, the proposed algorithm produces only admissible solutions, and is guaranteed to terminate.

We show that the following lemma holds.

Lemma 1: If an instance (C, O) of the cloud-edge allocation problem is solvable, i.e., it admits at least one solution, then for each component c in C there exists at least one offering o in O such that o satisfies the requirements of c .

Proof: Let (C, O) be a solvable instance of the cloud-edge allocation problem, and let $\langle P, \lambda \rangle$ be an admissible solution. Assume for the sake of contradiction that Lemma 1 does not hold, i.e., there exists a component c such that no offering can satisfy its requirements. This is in contradiction with the existence of an admissible solution. In fact, let $P_i \in P$

be the (only) deployment set containing c , and let $o = \lambda(P_i)$. Then, o is an offering satisfying the requirements of c . ■

With Lemma 1 in place, we can easily prove the solver's correctness.

Theorem 1 (Correctness): Given a solvable instance of the cloud-edge allocation problem (C, O) , Algorithm 1 produces an admissible solution.

Proof: To show that the solution (P, λ) returned by the algorithm is admissible, it suffices to show that P is a partition of C and that for each $p \in P$, $\lambda(p)$ is a compatible offering. By construction (see Algorithm 1), each component c can be added to at most one deployment set, so it holds that $DS[i] \cap DS[j] = \emptyset$ for each $i \neq j$, with $i, j \in \{1, \dots, n\}$. Moreover, each component c is necessarily added to a deployment set. Assume for the sake of contradiction that c could not be added to any deployment set. This means that the loop in lines 6–11 was completed without finding a compatible deployment set. This is, however, impossible since by Lemma 1, there exists at least one offering o satisfying the requirements of c , and the algorithm at some point must try to add c to an empty deployment set, so the call to `findOffering( $O, \{c\}$ )` is guaranteed to return a non-*nil* offering and c would be added to one of the partition subsets. Therefore, it holds that

$$\bigcup_{i=1}^n DS[i] = C$$

and P , which is defined as the set of nonempty potential deployment sets, is thus a partition of C .

Since by construction λ (see line 10 in Algorithm 1) only assigns compatible offerings to deployment sets, (P, λ) is an admissible solution. ■

D. Experimental Evaluation

To preliminarily evaluate the greedy approach we propose, we implemented it in the *R* programming language and ran it on randomly generated instances of the security-aware cloud-edge deployment optimization problem. In particular, we considered three different numbers of components (10, 30, 50) and three numbers of offerings (20, 50, 100), leading to nine different classes of instances, corresponding to the possible combinations. For each of these nine classes, we randomly generated ten different instances for a total data set amounting to 90 instances, identified by a name of the form X_Y_Z , where X is the number of components, Y is the number of offerings, and Z is the instance identifier in $\{1, \dots, 10\}$. These instances have been generated by an algorithm that was devised to generate plausible instances based on the actual public offerings of three top cloud providers. Consequently, the ownership costs of these instances depend on the hardware characteristics and the implemented security controls. For the on-premises offerings, we considered the reference costs for the total cost of ownership [33]. In all the above cases, the costs are expressed in a generic currency unit and do not refer to any particular currency. To achieve a baseline against which the solutions returned by our greedy algorithm could be compared, we computed a lower bound to the minimum cost and an admissible benchmark solution as explained below. In

TABLE I
EXPERIMENTAL RESULTS: THE COMPUTED COSTS

Instance	Greedy cost	$own_cost(s')$	$tot_cost(s')$	LP Time (hours)
10_20_2	1583	1388	1583	4,6
10_20_4	2933	2028	2933	2,9
10_20_5	1959	1295	1959	6,6
10_20_7	1848	1848	1848	3,0
10_20_8	2120	1410	2120	0,4
10_20_9	2899	2436	2899	70,8
10_20_10	1924	916	1924	9,8
10_50_1	2307	1903	2307	38,1
10_50_2	2780	2447	2999	16,0
10_50_3	452	270	270	4,3
10_50_4	2597	1444	2185	4,1
10_100_3	1489	685	1489	0,1
10_100_4	2215	2215	2215	27,5
10_100_5	2891	1969	2891	31,7

the optimal cloud-edge allocation problem, the cost function we seek to minimize is the sum of two components: 1) cost of ownership and 2) data transfer cost (see Section III-A4). If the data transfer cost component of the objective function is ignored, the resulting objective function is linear, and the problem can be solved using linear integer programming solvers (e.g., implementing the well-known simplex method), as suggested in [15]. Let $s' = \langle P', \lambda' \rangle$ be the solution, which can be computed as explained above, that minimizes the cost of ownership component. The minimum cost of ownership, i.e., $own_cost(s')$, that does not consider data transfer costs is a lower bound for the optimal cloud-edge allocation problem. In fact, no admissible solution can cost less than that since the data transfer cost is always nonnegative. Moreover, since s' satisfies all the problem constraints, it is an admissible solution for the security-aware cloud-edge allocation problem. Therefore, the total cost $tot_cost(s')$ is also an upper bound to the optimal cost of the cloud-edge allocation problem.

The tests were run on a Windows server equipped with an i7-4790 CPU and 24 GB of RAM, and the greedy algorithm produced a solution for each of the instances in less than 0.5 s.

We tried to solve each of the 90 randomly generated instances with both our greedy algorithm and a linear programming solver on a Windows server equipped with an i7-4790 CPU and 24 GB of RAM and allowed the solvers up to 72 h per problem instance to terminate. Under these conditions, our proposal produced a solution for each of the instances in less than 0.5 s, whereas we managed to find the solution minimizing the cost of ownership for only 14 of the 90 considered instances, all belonging to the smaller classes having ten components. For the other instances, the linear solver did not terminate in time. In Table I, we report the cost results for these 14 instances. In the table, the *greedy cost* column reports the total costs of the solutions found by our greedy algorithm. The $own_cost(s')$ and $tot_cost(s')$ columns report, respectively, the minimum admissible instance cost and the total cost of the solution s' that minimizes the instance cost. In the last column, we reported the time (in hours) that the linear solver took to find a solution. In the majority of these instances (11 out of 14), the solution found, in a fraction of the

time, by the greedy algorithm had a cost which was equivalent to the total cost of the solution that minimizes instance cost. As for the remaining three instances, in instance 10_50_2, the greedy algorithm found a solution which cost less than the one found by the linear solver, whereas in instances 10_50_3 and 10_50_4 class, it found a solution which costed more.

V. CONCLUSION AND FUTURE WORKS

In the IIoT scenario, the cloud-edge computational paradigm emerged as a valuable solution for many applications. Indeed, this paradigm distributes part of the computing and communication capabilities to the so-called edge devices. The latter may perform preliminary processing with very low latency and high-security standards, while heavy computation tasks can still be carried out in cloud data centers. On the other hand, this flexibility makes the well-known problem of finding an optimal allocation of software components to cloud and/or edge resources more difficult.

In this article, we proposed a novel formalization approach to model cloud-edge applications and their security-aware deployment over both cloud and on-premise resources, given specific performance, latency, and security constraints. While designing cloud-edge applications, developers may focus on different, often conflicting, objectives (e.g., minimizing costs, maximizing security, or maximizing performances). As a result, they often need to tradeoff one objective to favor another. To assist the developers in these choices, we defined an optimization strategy with three different objectives, depending on the specific goal of the application owner, namely: 1) minimize the overall solution cost; 2) maximize the solution security score; and 3) maximize the quality of the solution based on a multicriteria objective function over cost and security. In this article, we focused on the first of these three possible strategies.

Finding the optimal solution to minimize the overall cost can be very hard from a computational perspective due to the very large search space, even for small numbers of components and offerings. To address this issue, we proposed an efficient yet suboptimal deterministic algorithm that is able to compute a solution in polynomial time $O(n^2 \cdot (n + m))$, where n is the number of components and m the number of offerings. After proving the correctness of the proposed algorithm, we compared the quality of the achievable solutions against a solver based on linear programming. Given a data set of plausible components and cloud-edge offerings, our algorithm was able to find the same allocation solution of the linear solver while being many orders of magnitude faster (few tenths of a second versus many hours).

In future works, we would like to also consider multicriteria optimization functions and exploit different families of metaheuristic algorithms, such as the genetic one, to face the problem.

REFERENCES

- [1] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A survey," *Comput. Netw.*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [2] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A vision, architectural elements, and future directions," *Future Gener. Comput. Syst.*, vol. 29, no. 7, pp. 1645–1660, 2013.
- [3] O. Vermesan and P. Friess, *Internet of Things: Converging Technologies for Smart Environments and Integrated Ecosystems*. Aalborg, Denmark: River Publ., 2013.
- [4] B. P. Bhattarai *et al.*, "Big data analytics in smart grids: State-of-the-art, challenges, opportunities, and future directions," *IET Smart Grid*, vol. 2, no. 2, pp. 141–154, 2019.
- [5] M. Chen, S. Mao, Y. Zhang, and V. C. Leung, *Big Data: Related Technologies, Challenges and Future Prospects*. Cham, Switzerland: Springer, 2014.
- [6] *The Rise of Industrial Big Data*, GE, Charlottesville, VA, USA, 2012.
- [7] M. Baily and J. Manyika, *Is Manufacturing 'Cool' Again*, Project Syndicate, Prague, Czech Republic, 2013.
- [8] M. Marjani *et al.*, "Big IoT data analytics: Architecture, opportunities, and open research challenges," *IEEE Access*, vol. 5, pp. 5247–5261, 2017.
- [9] H. Xu, W. Yu, D. Griffith, and N. Golmie, "A survey on industrial Internet of Things: A cyber-physical systems perspective," *IEEE Access*, vol. 6, pp. 78238–78259, 2018.
- [10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [11] K. Kaur, S. Garg, G. S. Aujla, N. Kumar, J. J. Rodrigues, and M. Guizani, "Edge computing in the industrial Internet of Things environment: Software-defined-networks-based edge-cloud interplay," *IEEE Commun. Mag.*, vol. 56, no. 2, pp. 44–51, Feb. 2018.
- [12] Z. Á. Mann, "Allocation of virtual machines in cloud data centers—A survey of problem models and optimization algorithms," *ACM Comput. Surveys*, vol. 48, no. 1, pp. 1–34, 2015.
- [13] M. Rak, N. Suri, J. Luna, D. Petcu, V. Casola, and U. Villano, "Security as a service using an sla-based approach via SPECS," in *Proc. IEEE 5th Int. Conf. Cloud Comput. Technol. Sci. (CloudCom)*, vol. 2, Bristol, U.K., Dec. 2013, pp. 1–6.
- [14] E. Rios, E. Iturbe, L. Orue-Echevarria, M. Rak, and V. Casola, "Towards self-protective multi-cloud applications—MUSA—A holistic framework to support the security-intelligent lifecycle management of multi-cloud applications," in *Proc. 5th Int. Conf. Cloud Comput. Services Sci. (CLOSER)*, Lisbon, Portugal, May 2015, pp. 551–558.
- [15] V. Casola, A. De Benedictis, M. Eraqsu, J. Modic, and M. Rak, "Automatically enforcing security SLAs in the cloud," *IEEE Trans. Services Comput.*, vol. 10, no. 5, pp. 741–755, Sep./Oct. 2017.
- [16] V. Casola, A. De Benedictis, J. Modic, M. Rak, and U. Villano, "Per-service security slas for cloud security management: Model and implementation," *Int. J. Grid Utility Comput.*, vol. 9, no. 2, pp. 128–138, 2018.
- [17] B. Jennings and R. Stadler, "Resource management in clouds: Survey and research challenges," *J. Netw. Syst. Manag.*, vol. 23, no. 3, pp. 567–619, 2015.
- [18] S. S. Manvi and G. K. Shyam, "Resource management for infrastructure as a service (IaaS) in cloud computing: A survey," *J. Netw. Comput. Appl.*, vol. 41, pp. 424–440, May 2014, [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S1084804513002099>, doi: 10.1016/j.jnca.2013.10.004.
- [19] S. Singh and I. Chana, "Cloud resource provisioning: survey, status and future research directions," *Knowl. Inf. Syst.*, vol. 49, no. 3, pp. 1005–1069, 2016.
- [20] S. Singh and I. Chana, "A survey on resource scheduling in cloud computing: Issues and challenges," *J. Grid Comput.*, vol. 14, no. 2, pp. 217–264, 2016.
- [21] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta, and D. Sabella, "On multi-access edge computing: A survey of the emerging 5G network edge cloud architecture and orchestration," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1657–1681, 3rd Quart., 2017.
- [22] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1628–1656, 3rd Quart., 2017.
- [23] S. H. H. Madni, M. S. A. Latiff, Y. Coulibaly, and S. M. Abdulhamid, "Recent advancements in resource allocation techniques for cloud computing environment: A systematic review," *Clust. Comput.*, vol. 20, pp. 2489–2533, Dec. 2017.
- [24] V. Casola, A. De Benedictis, M. Rak, and U. Villano, "Security-by-design in multi-cloud applications: An optimization approach," *Inf. Sci.*, vols. 454–455, pp. 344–362, Jul. 2018.

- [25] T. Djemai, P. Stolf, T. Monteil, and J.-M. Pierson, "A discrete particle swarm optimization approach for energy-efficient IoT services placement over fog infrastructures," in *Proc. IEEE 18th Int. Symp. Parallel Distrib. Comput. (ISPD)*, 2019, pp. 32–40.
- [26] S. K. Mishra, D. Puthal, J. J. Rodrigues, B. Sahoo, and E. Dutkiewicz, "Sustainable service allocation using a metaheuristic technique in a fog server for industrial applications," *IEEE Trans. Ind. Informat.*, vol. 14, no. 10, pp. 4497–4506, Oct. 2018.
- [27] S. K. Battula, S. Garg, R. K. Naha, P. Thulasiraman, and R. Thulasiram, "A micro-level compensation-based cost model for resource allocation in a fog environment," *Sensors*, vol. 19, no. 13, p. 2954, 2019.
- [28] A. R. Benamer, H. Teyeb, and N. B. Hadj-Alouane, "Latency-aware placement heuristic in fog computing environment," in *Proc. OTM Confederated Int. Conf. Move Meaningful Internet Syst.*, 2018, pp. 241–257.
- [29] S. Dehnavi, H. R. Faragardi, M. Kargahi, and T. Fahringer, "A reliability-aware resource provisioning scheme for real-time industrial applications in a fog-integrated smart factory," *Microprocess. Microsyst.*, vol. 70, pp. 1–14, Oct. 2019.
- [30] D. Mourtzis, E. Vlachou, and N. Milas, "Industrial big data as a result of IoT adoption in manufacturing," *Procedia CIRP*, vol. 55, pp. 290–295, Oct. 2016.
- [31] National Institute of Standards and Technology. *Special Publication 800-53 Revision 4: Security and Privacy Controls for Federal Information Systems and Organizations*. Accessed: Jun. 20, 2020. [Online]. Available: <http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r4.pdf>
- [32] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*. Boston, MA, USA: Springer, 1972, pp. 85–103.
- [33] AWS. *AWS TCO Calculator*. Accessed: Jun. 20, 2020. [Online]. Available: <https://awstccalculator.com/>



Valentina Casola received the Ph.D. degree in computer engineering from the Second University of Naples, Caserta, Italy, in 2004.

She is an Associate Professor with the Department of Electrical Engineering and Information Technology, University of Naples Federico II, Naples, Italy. She has published more than 100 papers in journals, conference proceedings, and books. These activities are led in cooperation with academic institutions and industrial partners within national and international projects. Her

research activities are both theoretical and experimental and focus on security methodologies to design and evaluate distributed systems, including cyber-physical infrastructures and cloud systems and Web services.



Alessandra De Benedictis received the Ph.D. degree in computer and automation engineering from the University of Naples Federico II, Naples, Italy, in 2013.

She is an Assistant Professor with the Department of Electrical Engineering and Information Technology, University of Naples Federico II. Her research interests include the design and evaluation of secure architectures for the protection of distributed resource-constrained devices and the development of methodologies for

the security analysis and assessment of complex applications in IoT and cloud environments, and according to security-by-design approaches.



Sergio Di Martino received the M.Sc. and the Ph.D. degrees in computer science from the University of Salerno, Italy, in 2001 and 2005, respectively.

He is an Associate Professor with the Department of Electrical Engineering and Information Technology, University of Naples Federico II, Naples, Italy, where he is also the Co-Chair of the Knowledge Management and Engineering Lab. He has published more than 100 papers in journals, conference proceedings, and books. His research

interests focus on management and analytics of complex and massive data sets, software engineering, and software architectures.



Nicola Mazzocca received the M.Sc. degree in electronic engineering and the Ph.D. degree in computer engineering from the University of Naples Federico II, Naples, Italy, in 1987 and 1991, respectively.

He is currently a Professor of high-performance and reliable computing with the Computer and System Engineering Department, University of Naples Federico II. He authored over 200 papers in international journals, books, and international conferences in the field of computer architecture, reliable and secure systems, distributed systems, and

performance evaluation in high-performance systems.



Luigi Libero Lucio Starace received the M.Sc. degree (*cum laude*) in computer science from the University of Naples Federico II, Naples, Italy, in 2018, where he is currently pursuing the Ph.D. degree with the Information Technology and Electrical Engineering.

His research interests include the design and evaluation of end-to-end testing techniques, automating test generation and prioritization, formal methods, and metaheuristic approaches to solve optimization problems.